Redrob Ranker

Intelligent Candidate Discovery & Ranking — India.Runs (Redrob × Hack2Skill)

Team Name: Alonecoder

Team Leader: Karthikeyan M

Problem Statement: From a 100,000-candidate pool, rank the top 100 for a Senior AI Engineer role — by understanding who actually fits, not by matching keywords.

Solution Overview

What we built — a hybrid ranker that combines three lenses:

- Semantic understanding (embeddings) — reads each candidate's career story.
- Interpretable recruiter rules — real role, production evidence, trajectory.
- Behavioral availability + trap/honeypot guards.

What differentiates us from traditional matching:

- We proved the skills list is noise (every skill appears ~12,000 times, uniformly sprayed). We ignore it and read the career narrative instead.
- We model the JD's explicit anti-patterns (research-only, computer-vision, consulting-only, title-chasers) as negative signals — keyword/skill matchers cannot do this and get trapped.

JD Understanding & Candidate Evaluation

Key requirements extracted from the JD:

- Production embeddings/retrieval, vector DB / hybrid search, ranking evaluation (NDCG/MRR/MAP), strong Python; 5-9 yrs at product (not services) companies; Pune/Noida or willing to relocate; active on the platform.
- Explicit disqualifiers: pure research with no production; <12mo "LangChain-calls-OpenAI"; title-chasers; consulting-only careers; CV/speech without NLP/IR.

Signals that most determine relevance (beyond keywords):

- Current and past job TITLE; career-history free-text showing real production retrieval/ranking work; product-vs-services trajectory and tenure stability; behavioral availability. We judge fit on the career story, not the skills array.

Ranking Methodology

How we retrieve, score, and rank:

- Embed each profile's career text; score against the JD split into positive AND negative facet queries; add interpretable feature scores; multiply by an availability modifier; sort; take the top 100.

Models / algorithms / heuristics:

- Sentence-transformer (all-MiniLM-L6-v2) cosine similarity; rule-based role classifier; regex evidence extraction; gaussian experience-band; honeypot impossibility checks.

How signals combine into one score:

- Weighted sum: role 0.34 + semantic 0.30 + evidence 0.22 + band 0.08 + location 0.06, times a behavioral multiplier (0.55-1.08). Honeypots are forced to zero.

Explainability & Data Validation

How ranking decisions are explained:

- Every candidate gets a 1-2 sentence reason citing real facts: title, company, years, the specific evidence found, and key signal values.

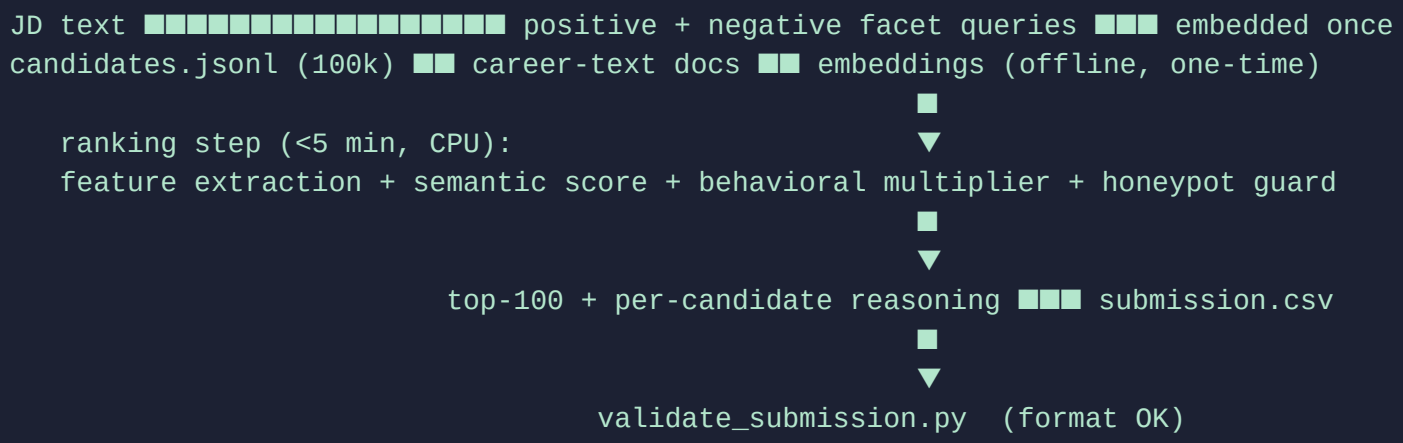
How we prevent hallucination / unsupported justifications:

- Reasons are generated programmatically from the exact features that drove the score — nothing is invented, and each one names an honest concern.

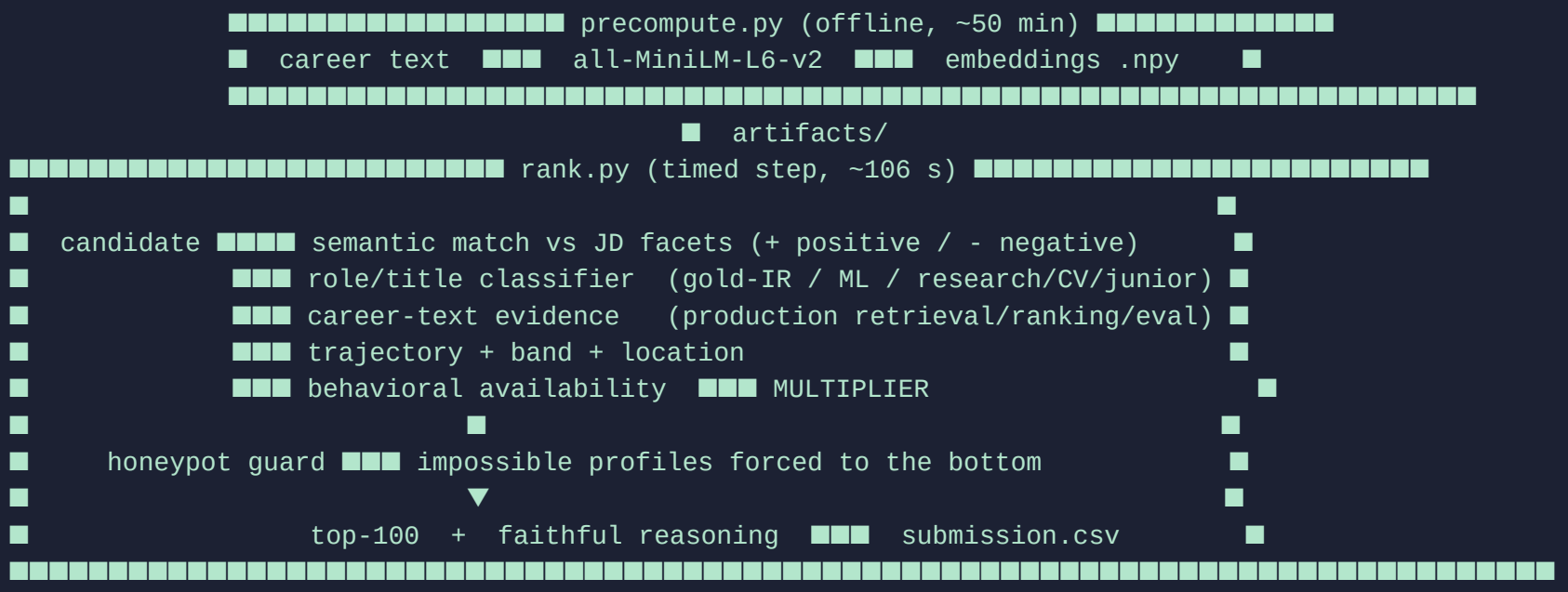
How we handle inconsistent / suspicious / low-quality profiles:

- A strict impossibility guard (job duration exceeding time since it started; many "expert" skills with 0 months used; end-before-start dates) forces the ~80 planted honeypots to the bottom — 0 of them reach our top 100.

End-to-End Workflow



System Architecture



Results & Performance

Ranking quality on the 100k pool:

- Top-100 = 53 recommendation/search/applied-ML engineers + 47 ML engineers.
- 0 keyword-stuffers, 0 research/CV/junior traps, 0 honeypots in the top 100.
- Top-10 all show production retrieval + ranking-evaluation evidence, 5-8 yrs, based in Indian tech hubs; 100 distinct scores (elite tier differentiated).

Meeting the runtime / compute constraints:

- Ranking step runs in ~106 s (limit: 5 min), < 16 GB RAM, CPU-only, no network and no LLM calls. The heavy embedding is a one-time offline precompute.

Technologies Used

- Python 3.11 — core.
- sentence-transformers (all-MiniLM-L6-v2) — small, CPU-fast, GPU-free embeddings.
- NumPy — fast vector math for the timed ranking step.
- uv — reproducible dependency management.
- Gradio — hosted sandbox demo.
- reportlab — this deck.

Why: every choice respects the CPU / 5-minute / no-network budget, keeps the system fully reproducible offline, and stays transparent enough to defend in the Stage-5 interview.

Submission Assets

- GitHub repository — full source, README, CLAUDE.md, reproduce command.
- submission.csv — the ranked top-100 candidates.
- This deck (PDF).
- Live sandbox — HuggingFace Spaces demo (ranks an uploaded sample).
- submission_metadata.yaml — team + compute + AI-tools declaration.

Thank You

Redrob Ranker — Build what next India runs on